

"How the Internet Stalks You"

Part 1: The Hook, The Myth & The Tech Shift

Intro & Story

Hello everyone. Before we start, please take a look at the screen. You'll see a QR code. This links to our digital handout, you maybe have to open through the download folder.

It contains all the key terms we'll discuss today. Please take a moment to scan it now. *[Pause for scanning]*.

Great. Now, let me ask you something. How many of you keep your phone with you at all times?

[Pause and look at the audience]

Even in the bathroom, right?

Let me ask a second question: Raise your hand if you've ever had a conversation about a product — maybe sneakers, a holiday, or food — and just ten minutes later, you saw an ad for it on Instagram or TikTok? *[Pause and look at the audience]*.

Exactly. It feels like your phone is listening, right? Well, let me tell you what happened to me recently. We were meeting for a group project at a classmate's apartment. During a break, two of my group members started chatting about their shared passion: horse riding. I wasn't involved in that conversation at all. Honestly, horses aren't my thing, so I didn't listen, I didn't search for anything related to riding, and I didn't say a single word about it.

But later that evening, back at home, I was scrolling through Instagram, and suddenly—my feed was flooded with ads for saddles and horse gear.

That really creeped me out. I was so confused that I started researching how on earth that could possibly happen.

Was my phone actually listening to our conversation? Probably not.

But the truth is almost as scary. Today, we want to show you how the internet stalks you and why it is legal *[Pause]* mostly.

Outline

Let's first look at the outline of our presentation.

First I will introduce you to `The Myth` .

After that we will talk about `The Toolkit` as well as the increasing `Architecture Shift` .

Furthermore, Luca will tell you how `Geo Fencing & Offline Tracking` works.

And he will explain to you why it is legal.

After a brief `Conclusion` there will be also `Room for discussion` .

The Myth: Predictive Analytics

Let's start with the myth.

People love to blame the microphone for these creepy ads.

It's a **common and convenient assumption** because it makes us feel like we're just victims of a technical glitch.

But the reality isn't a glitch — it's Predictive Analytics.

Why would they even need to listen to your voice when they **already have your age, your location, your friends, and your entire browsing history?** *[Pause for 2-3 sec]*

These algorithms don't read your mind; they calculate patterns.

They know that people with your exact profile

- hanging out with your friends
- living in your area

often start developing the same specific interests.

They aren't predicting the future; they are simply calculating the probability of your next move.

They know what you want to buy before you even have the conscious thought.

This certainly feels like stalking — but at the end of the day, it's just **highly efficient, automated targeting**.

For instance I found an **Poll on Twitter with about 234 votes**.

80% of the people are **claiming that their phones are listening** to them.

The **fun fact** of this outcome is that most of the **initiators followers and retweets** came from the **information security community**.

The Toolkit

Let's move on to the toolkit.

So, how do they get all this data?

Cookies:

Let's start with the classic: Cookies. And unfortunately, not the tasty kind.

As you all know, **HTTP is a stateless protocol**.

To maintain the state, we need a mechanism to persist information across requests.

That's where cookies come in.

Technically, it's a small piece of data — a `key-value pair` — that the server issues via the `Set-Cookie` header.

Your **browser stores** this and **sends it back in the Cookie request header** for every subsequent request to that domain.

This allows the **server to map incoming HTTP requests to a specific User-ID** - a so called UID.

Essentially, it bridges the gap between stateless requests, turning them into a persistent user session.

This UID is the **foundation of traditional online tracking**.

It's how the server links your activity from Monday to Tuesday and reconstructs your user journey.

But relying on the client to manage so-called **Third Party Cookie** is becoming problematic since they are set through external domains.

With stricter **privacy controls in modern browsers**, **storing persistent identifiers on the clientside is being heavily restricted**.

Sometimes, the infrastructure needs to **trigger this state more indirectly**.

That's why companies are **increasingly relying on server-side tracking** these days, where **First Party Cookies** are set by the company domain itself.

But I'll get to that later.

Tracking Pixels:

Now, how is this data actually exfiltrated? Let's look at Tracking Pixels.

Technically, these are **minimal DOM elements** — often a 1x1 transparent GIF or a small script **embedded in the page**.

The trigger mechanism is simple: when the browser renders the page, it automatically executes an `HTTP GET request` to the advertiser's server.

What makes this powerful for tracking is the payload of that request:

- First of all the **Request Headers**: By default, the **browser sends your User-Agent, your IP address, and — crucially — the Referrer header**, which tells the advertiser exactly which URL you are currently visiting.
- And of course the **Cookie Synchronization**: If your browser already holds a third-party cookie for that advertiser's domain, the browser attaches it to the request header.

This effectively **bridges your current session with your long-term profile on their backend**. It's a **distributed event-logging system** that allows the advertiser to **reconstruct your user journey in real-time**.

This is the primary mechanism that enables **cross-site tracking** and **third-party cookie syncing**.

Browser Fingerprinting:

Finally, we have the most robust method for identification: Browser Fingerprinting.

Unlike cookies, which rely on stateful storage, **fingerprinting is stateless**.

It's based on the premise that the **aggregate configuration of a user's environment**

- the browser
 - the operating system
 - and the hardware
- is **statistically unique**.

The tracker executes a **series of JavaScript API queries** to **collect high-entropy data points**:

- First example is the **Canvas Fingerprinting**:
 - It forces your browser to **render a hidden graphic**.
 - Because every
 - graphics card
 - driver version
 - and font-rendering engine
 - **processes sub-pixel antialiasing slightly differently**, the resulting image hash is statistically unique to your machine.
- Near to this there is the **AudioContext API**:
 - It doesn't record audio, but it **tests the hardware's audio stack**.
 - By generating a **sound wave and measuring how your hardware processes that signal**, it identifies differences in your audio drivers and signal processing.

- Also we have the **Web Graphics Library** (short: WebGL) and the **Environment Enumeration**:
 - It probes your **graphics processing unit's performance via WebGL** and enumerates
 - installed fonts,
 - plugins
 - and system settings.

But an **important restriction every company must consider** for all of these 3 points:

It's only allowed **after user consent**, in this case advertisers always simply use cookies.

The secret service is of course not obligated to give its permission.

According to fingerprinting, here is a critical point for the engineers:

Many assume that privacy tools like Ad Blockers, VPNs or the Tor Browser offer protection here.

[Pause]

They don't.

We **haven't researched this in full depth**, as it would be too much overhead for this presentation, but we still want to **briefly explain the key points of my statement**.

An **Ad Blocker** blocks network requests to known tracker domains, but fingerprinting is executed locally within your browser context.

[Pause]

And a **VPN** operates at Layer 3 - the network layer of the OSI layer model

It tunnels your traffic and masks your IP, but fingerprinting operates at Layer 7 - the application layer .

Using a VPN to stop fingerprinting is like **driving a car with a different license plate while keeping your engine serial number exactly the same**.

The algorithm knows it's still you.

Your **IP is just one variable; the device itself**

- the hardware rendering,
 - the driver-specific behavior,
 - the font list
- remains constant.**

Furthermore, you should **check whether the VPN provider is secure**.

Otherwise, data brokers can easily find out that you have paid for a VPN and are most likely using one.

For instance **Mullvad VPN** offers a special payment method for this.

You **put the €5 fee in an envelope and send it to the company by mail**.

But be careful: sending money by mail is also insecure... but that's a different story.

On the other hand, we have the **problem that despite using a VPN that encrypts payment data via SSL or TLS**, we have no control over the fact that **payment providers like Paypal sell our data**.

[Pause]

So what about the **Tor browser**?

It's an open-source project and based on Firefox, but it **routes all requests through various servers** — so-called **Tor nodes** — to obfuscate the IP address.

To **protect users from identification via browser fingerprinting**, the Tor Browser **appears the same** to the requested website,

- regardless of the platform it's running on,
- including details like browser version
- and operating system.

Therefore, it's **best not to personalize** the Tor Browser at all.

Furthermore, the **total number of available nodes significantly influences how difficult it is** (even for authorities) to **monitor** a sufficiently large portion of the decentralized Tor infrastructure for **timing analysis**.

With roughly **7,000 to 8,000 Tor nodes**, the network has **barely grown in the recent years**.

These figures can be viewed on The Tor Project's website. I can send a link to it if you're interested.

[Pause]

Just a small ray of hope for all **Linux users**.

This operating system is actually somewhat **more secure** because the **source code can be viewed**, and **security vulnerabilities** can be at least identified by yourself, if you have the necessary knowledge.

"Security by obscurity", as is the case with **Windows**, doesn't work and only represents **another security vulnerability**.

The Architecture Shift (Server-Side Tracking)

So, if Ad-blockers and browser privacy protections are getting better at blocking these identifiers, as I mentioned before, why is tracking still so effective?

Here is the plot twist: The Shift from clientside to serverside Tracking.

Historically, tracking was **clientside**. The **JavaScript tracker code** ran in your browser, and your **browser sent the data directly to Google** or Facebook's servers.

Ad-blockers could easily 'see' these requests and drop them.

Now, companies have moved to **serverside Tracking**.

Instead of your browser talking to Facebook, your **browser sends the data to the company's own server** (your 'first-party' domain).

The data is then **processed on their backend and forwarded to the ad-network servers** via a server-to-server (S2S) call.

Why this matters:

- On the one hand **Invisibility**: Since the request happens **server-to-server**, there is no outgoing network request from your browser to an ad-network that your **Ad-blocker could intercept**.
- On the other hand **Technique-Agnostic**: Because the processing happens on the server, the **server doesn't care how you were identified**. Whether they used a cookie, a user-login, or a browser fingerprint hash — the server can package this identifier and send it to the ad-network **without your browser ever knowing the data left your machine**.

In short:

Tracking has moved from the 'public' client-side, where you could see and block it, to the 'private' server-side, where it is effectively invisible.

Transition

So, **the digital side is becoming nearly impossible to escape**. But what happens when you put the laptop away and walk out the door? Luca will explain that.

Conclusion & Interaction

To sum it up: You are the product.

- Every click,
- every step,

- every location

is data that is bought and sold.

It's a constant game of cat and mouse between trackers and privacy tools.

Before we finish: We have two final questions:

- Does anyone have a specific example of when they felt stalked by their phone?
- How many of you use ad blockers? And did you think they really helped? [Wait for answers]
 - The truth is: They **help against basic ads**, but they struggle against serverside Tracking.
 - The **technology is always one step ahead**.